



Budget Tracker — Documento di progetto



Obiettivo del progetto

App personale di monitoraggio del budget che si collega al conto (Hype, via aggregatore Tink), importa automaticamente le transazioni, le categorizza, confronta la spesa con le soglie definite e invia notifiche quando una categoria si avvicina o supera il limite.

1. Analisi funzionale

Cosa fa il sistema, per chi e con quali flussi.

1.1 Attori

- **Utente** — definisce categorie e soglie, consulta la dashboard, riceve notifiche.
- **Sistema** — importa transazioni, categorizza, aggrega, valuta le soglie, notifica.
- **Aggregatore bancario (Tink)** — fornisce le transazioni della banca collegata.

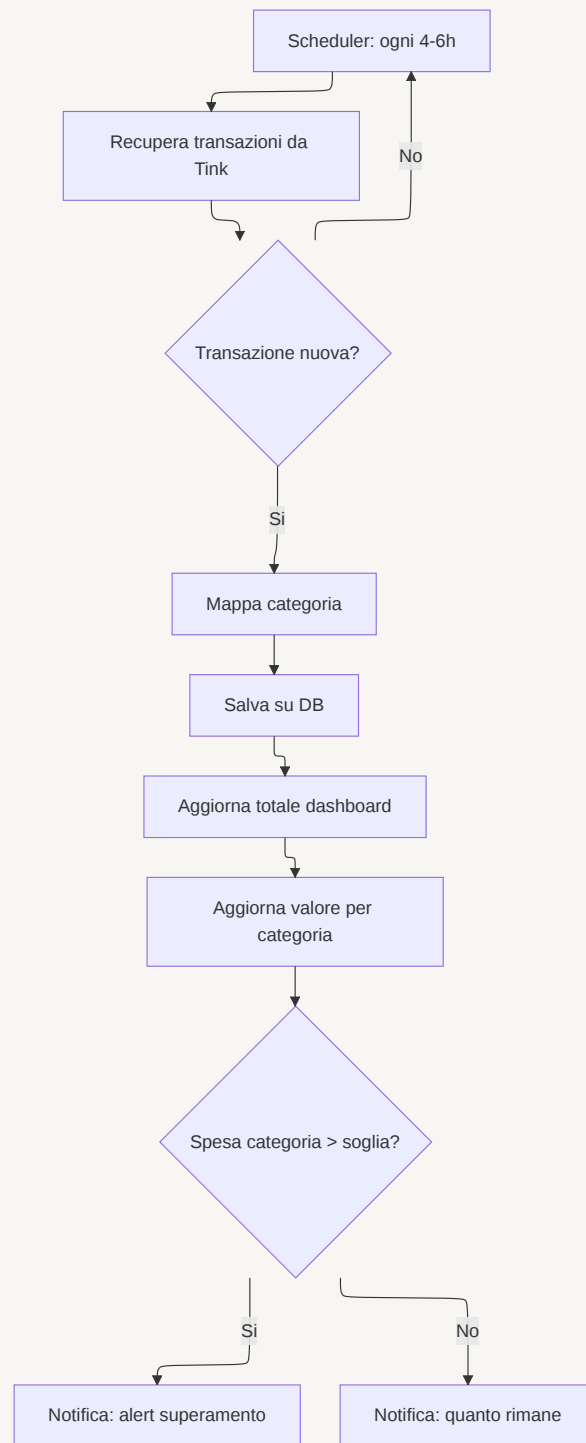
1.2 Funzionalità principali

1. **Collegamento conto** — autenticazione una tantum verso la banca tramite Tink (consenso Open Banking).
2. **Import transazioni** — recupero periodico delle nuove transazioni e deduplica rispetto a quelle già salvate.
3. **Categorizzazione** — assegnazione di una categoria a ogni transazione (categoria Tink di default, rimappabile su categorie personali).
4. **Definizione budget** — l'utente imposta una soglia di spesa per categoria (mensile).

5. **Aggregazione e dashboard** — calcolo del totale speso e dello speso per categoria nel periodo corrente.
6. **Valutazione soglie** — confronto spesa/soglia per categoria.
7. **Notifiche** — alert al superamento soglia; promemoria del residuo quando si è sotto soglia.

1.3 Flusso principale (happy path)

```
flowchart TD
    A["Scheduler: ogni 4-6h"] --> B["Recupera transazioni da Tink"]
    B --> C{"Transazione nuova?"}
    C -- No --> A
    C -- Si --> D["Mappa categoria"]
    D --> E["Salva su DB"]
    E --> F["Aggiorna totale dashboard"]
    F --> G["Aggiorna valore per categoria"]
    G --> H{"Spesa categoria > soglia?"}
    H -- Si --> I["Notifica: alert superamento"]
    H -- No --> J["Notifica: quanto rimane"]
```



1.4 Requisiti non funzionali

- **Affidabilità import:** nessuna transazione duplicata, nessuna persa (deduplica robusta).
- **Costi:** stack a costo zero o minimo per uso personale.
- **Privacy:** dati bancari trattati solo localmente / su servizio controllato dall'utente.
- **Manutenibilità:** logica di categorizzazione e soglie facilmente modificabile.

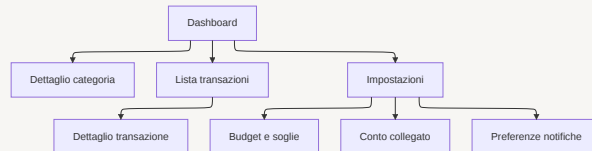
2. Information Architecture dell'interfaccia

Struttura delle schermate, navigazione e contenuti.

2.1 Mappa delle schermate

```

flowchart TD
    Home["Dashboard"] --> Cat["Dettaglio categoria"]
    Home --> Tx["Lista transazioni"]
    Home --> Set["Impostazioni"]
    Set --> Bud["Budget e soglie"]
    Set --> Conn["Conto collegato"]
    Set --> Notif["Preferenze notifiche"]
    Tx --> TxDet["Dettaglio transazione"]
  
```



2.2 Contenuto per schermata

Schermata	Contenuto chiave	Azioni
Dashboard	Totale speso nel mese, residuo complessivo, barre per categoria (speso/soglia), stato (ok / vicino / superato)	Filtro periodo, tap su categoria
Dettaglio categoria	Speso vs soglia, trend, elenco transazioni della categoria	Modifica soglia, ricategorizza transazione
Lista transazioni	Transazioni recenti con data, importo,	Cerca, filtra, apri dettaglio

Schermata	Contenuto chiave	Azioni
	categoria, merchant	
Budget e soglie	Categorie con relativa soglia mensile	Crea/modifica/elimina categoria e soglia
Conto collegato	Stato connessione Tink, ultimo sync	Collega/ricollega, forza sync
Preferenze notifiche	Canale (Telegram/email), soglia di alert, frequenza promemoria	Attiva/disattiva, configura canale

2.3 Principi di navigazione

- **Dashboard come hub:** tutto parte da lì; massimo 2 tap per arrivare a qualsiasi informazione.
- **Gerarchia chiara:** panoramica → categoria → transazione.
- **Impostazioni separate** dal flusso di consultazione quotidiana.

3. Architettura software

Componenti, dati e integrazioni.



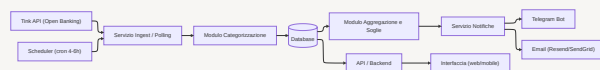
Scelte tecniche V1: dashboard web su Supabase · database Postgres gestito da Supabase · backend Node.js.

3.1 Diagramma dei componenti

```

flowchart LR
    Tink["Tink API (Open Banking)"] --> Ingest["Servizio Ingest / Polling"]
    Cron["Scheduler (cron 4-6h)"] --> Ingest
    Ingest --> Cat["Modulo Categorizzazione"]
    Cat --> DB["Database"]
    DB --> Agg["Modulo Aggregazione e Soglie"]
    Agg --> Notif["Servizio Notifiche"]
    Notif --> TG["Telegram Bot"]
    Notif --> Mail["Email (Resend/SendGrid)"]
    DB --> API["API / Backend"]
    API --> UI["Interfaccia (web/mobile)"]

```



3.2 Componenti

Componente	Responsabilità	Tecnologia consigliata
Scheduler	Avvia il polling ogni 4-6 ore	cron job
Ingest / Polling	Chiama Tink, confronta con lo storico, deduplica	Node.js
Categorizzazione	Mappa categoria Tink → categoria personale	Logica applicativa + tabella di mapping
Database	Transazioni, categorie, soglie, stato sync	Supabase / Postgres
Aggregazione e Soglie	Calcola totali, confronta con soglie	Query aggregate sul DB
Notifiche	Invia alert e promemoria residuo	Telegram bot (gratis) o email
Backend / API	Espone dati all'interfaccia	Node.js o FastAPI
Interfaccia	Dashboard e gestione budget	Web (Supabase) o app mobile

3.3 Modello dati (essenziale)

- **transaction** — `id`, `external_id` (per deduplica), `date`, `amount`, `merchant`, `category_id`, `raw_category` (da Tink).
- **category** — `id`, `name`, `monthly_threshold`.
- **budget_period** — `id`, `category_id`, `period` (mese), `spent`, `threshold`.
- **sync_state** — `last_sync`, `cursor/last_external_id`.

3.4 Integrazione bancaria

- **Aggregatore: Tink (by Visa)** — supporta Hype in Italia (`it-hype-ob`), piano gratuito per sviluppatori.^[1]
- **Nota su GoCardless:** scartato perché il free tier per nuovi utenti è diventato difficile da attivare (signup spesso reindirizzato ai piani a pagamento).^[1]

- **Meccanismo:** polling periodico (webhook se disponibile); confronto tra transazioni salvate e nuove.

3.5 Stack minimo consigliato

- **Backend:** Node.js
 - **DB:** Supabase / Postgres
 - **Aggregatore:** Tink
 - **Notifiche:** Telegram bot (zero costi, setup rapido)
 - **Scheduling:** cron ogni 4-6 ore
-

4. Prossimi passi

- ☐ Validare l'accesso al piano sviluppatori Tink e il collegamento Hype
- ☐ Definire le categorie personali e le soglie mensili
- ☐ Impostare la struttura del progetto (backend + DB)
- ☐ Implementare ingest + deduplica
- ☐ Implementare aggregazione + valutazione soglie
- ☐ Configurare il bot Telegram per le notifiche